INSTRUCTOR-LED PROGRAM • 48 HOURS • 12 HALF-DAY MODULES

DAY 2 / MODULE 2

AWS Fundamentals for the Workshop

IAM, S3, Lambda, CloudWatch, and event-driven design — the AWS layer your agentic prompts will land on for the rest of this program.

Prepared for **Accelya Kale** • Delivered by **Jai Singh**

What you shipped yesterday — and what it becomes today

Day 2 doesn't start from zero. Every artefact you committed to your GitLab branch becomes the input for a real AWS build.

YESTERDAY
`verify_bedrock.py`

↓ *becomes* ↓

TODAY
the CLI/runtime pattern you'll reuse while deploying Lambda today.

YESTERDAY
`/prompts/m1-arch-v1.md`

↓ *becomes* ↓

TODAY
the basis for your S3 zone design prompt in Lab 2.

YESTERDAY
`/outputs/m1-arch-options.json`

↓ *becomes* ↓

TODAY
the architecture you actually deploy in Lab 3.

YESTERDAY
`Decision MR (Option A)`

↓ *becomes* ↓

TODAY
the IAM role + Lambda + S3 layout you'll instantiate today.

YESTERDAY
`HITL_REVIEW.md (chosen option)`

↓ *carries forward* ↓

TODAY
the dimensions you'll re-score against running Lambda code.

YESTERDAY
`Airline vocabulary (12 terms)`

↓ *reappears in* ↓

TODAY
Lambda payloads, IAM tags, CloudWatch log fields, S3 prefixes.

Day 2 agenda

Same rhythm as Day 1: concepts → guided design → hands-on labs → review wrap.

1 — 30–35 min

Concept briefing

AWS service map for this program. IAM essentials. S3 fundamentals (zones). Lambda fundamentals. CloudWatch. Event-driven design.

2 — 35–40 min

Guided design

S3 zone model walkthrough (raw → validated → curated). Permissions boundaries. Lambda handler anatomy. Structured logging template.

3 — 2 – 2¼ hrs

Hands-on labs

(1) AWS sandbox sign-in + IAM verify. (2) S3 zones inspection + setup. (3) Deploy starter Lambda + read CloudWatch logs.

4 — 20–30 min

Review, Q&A, wrap

Module 2 takeaways. IAM least-privilege mistakes. End-of-module cleanup. Day 3 preview.

Day 2 work commits to a new branch: module-2-<your-name>. Day 1 artefacts are referenced from module-1-<your-name>, not re-committed.

BY THE END OF TODAY YOU WILL BE ABLE TO

Day 2 learning outcomes

READ

Read an IAM policy and explain what it grants, denies, and bounds for an airline data role.

CONFIGURE

Set up an S3 zone layout (raw / validated / curated / quarantine) for ticket-issue ingestion.

UPDATE

Package and update a Python Lambda from your branch via the AWS CLI — from local code to running function.

INSPECT

Trace a single invocation through CloudWatch logs using a correlation ID.

DEBUG

Diagnose a failed Lambda execution from logs alone — IAM, payload shape, timeout, or memory.

REUSE

Apply the HITL checklist to a real Lambda and document the IAM least-privilege decisions.

These outcomes feed directly into Module 3, where the Lambda and S3 layout you build today gets wrapped in agentic prompts.

01

PART 1 · ~30–35 MINUTES

Concept briefing

Five AWS services. One job per service. By the end of this section you will know exactly which service to reach for when designing the rest of the program’s pipelines.

AWS service map for the workshop

Five services do 90% of the work in this program. Know what each one is for; the rest is composition.

IAM

Identity & permissions

Who can do what — users, roles, policies, boundaries.

AIRLINE

An ingestion role that can read raw/, write validated/, never read PII.

Modules 2–12

S3

Object storage

Buckets and prefixes for raw, validated, curated, quarantine zones.

AIRLINE

Daily ticket-issue files arrive in raw/ via SFTP gateway.

Modules 2–10

Lambda

Event-driven compute

Function triggered by events; pay per invocation; no servers to manage.

AIRLINE

On S3 PUT, validate the file and route it to validated/ or quarantine/.

Modules 2–11

CloudWatch

Logs, metrics, alarms

Where Lambda writes logs and where you read them; metrics + alarms on top.

AIRLINE

Correlation ID per file; alert on quarantine/ growth above threshold.

Modules 2–12

EventBridge / S3 Events

Event plumbing

How services tell each other something happened.

AIRLINE

S3 PUT → Lambda; cancellation event → Lambda; daily schedule → Lambda.

Modules 4–10

Today: get familiar with all five. Tomorrow onwards: combine them under agentic prompts to build real airline pipelines.

IAM essentials — four primitives, one rule

AWS access boils down to four building blocks. The rule on top is always the same: least privilege.

01

Identity

Who is asking?

Trainer-issued IAM user, or a Lambda execution role.

02

Policy

What is allowed?

JSON document: actions, resources, conditions.

03

Role

Temporary credentials

Lambda assumes a role at runtime; no long-lived keys.

04

Boundary

What is the ceiling?

Permission boundary caps what any policy can grant.

THE RULE — LEAST PRIVILEGE

Grant exactly what the task needs. Nothing more. If a Lambda only needs to read raw/ and write validated/, the role gets s3:GetObject on raw/* and s3:PutObject on validated/* — not s3:* on the bucket.

S3 fundamentals — buckets, prefixes, events

The five things you need to know about S3 before designing zones in Part 2.

FIVE BUILDING BLOCKS

Bucket

Globally unique name. One bucket can hold any number of objects.

Prefix

Folder-like path inside a bucket: raw/, validated/, curated/, quarantine/.

Object

The file itself + metadata + size + storage class. Up to 5 TB per object.

Event

PUT, DELETE, multipart-complete — fired to Lambda, SQS, EventBridge, SNS.

Versioning + lifecycle

Keep history; transition cold data to cheaper classes; expire after retention.

AIRLINE ZONE PREVIEW — DETAIL IN PART 2

raw/
What the airline gave us. Untouched. Audit-grade.

validated/
Schema-checked, PNR/airline_code/fare_class present.

curated/
Partitioned Parquet. Athena-ready. Joins applied.

quarantine/
Bad records. Reason logged. Reviewed by ops.

Lambda fundamentals — the function, the runtime, the role

A Lambda is just a function with an event, a context, and an IAM identity. The five settings below decide its behaviour and cost.

handler.py # python3.13 · runtime: provided by AWS

```python
import json, logging, os

log = logging.getLogger()
log.setLevel("INFO")

def lambda_handler(event, context):
    # event = the trigger payload (S3, EventBridge, ...)
    # context = invocation metadata, request_id, timeout left
    log.info("received", extra={"corr_id":
context.aws_request_id})
    return {"statusCode": 200}
```

Bottom line: a Lambda is your code + a runtime + an IAM role. The rest is just configuration.

Runtime	python3.13 — AWS-supported AL2023 runtime used for cohort consistency; python3.14 also available.
Memory	128 MB → 10 GB. Memory also scales CPU. Lab default: 256 MB.
Timeout	Up to 15 minutes. Lab default: 30 seconds. Tune to expected work.
IAM role	Execution role assumed at startup; defines what the function can do.
Cold start	First invocation in a fresh env is slower. Mitigated by reuse + provisioned concurrency.

CloudWatch — where Lambda becomes observable

Three things CloudWatch does for you. Today is mostly the first one. Modules 5 and 11 will lean harder on the other two.

01 Log groups & streams

WHAT IT IS

Every Lambda writes to /aws/lambda/<name>. One log group per function; one stream per execution environment.

WHEN YOU NEED IT

Where you read what your function did. The bread and butter of debugging.

02 Metrics

WHAT IT IS

Built-in metrics (Invocations, Errors, Duration, Throttles) + custom EMF metrics from your code.

WHEN YOU NEED IT

Aggregate view: how often, how slow, how broken. Dashboards live here.

03 Alarms

WHAT IT IS

Threshold rules on metrics (e.g. error rate > 1% over 5 min). Trigger SNS, Lambda, or just notify.

WHEN YOU NEED IT

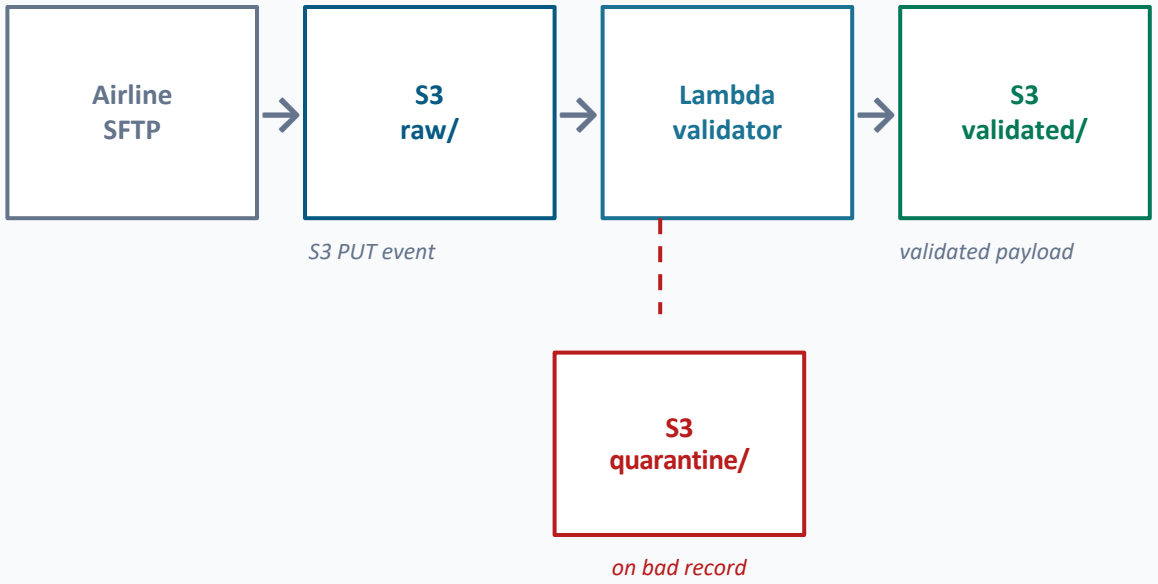
Wake someone up when something's wrong. Today: just be aware they exist.

Habit from Day 1: structured JSON logs with a correlation_id, plus airline-aware fields (records_total, airline_code, zone). Plain print() is allowed in dev; banned in this course from Module 5 onwards.

Event-driven design — push, don't pull

Most of what you'll build in this program is a chain of events. Three principles keep those chains reliable.

A TYPICAL FLOW



CloudWatch logs and metrics emit at every step. Prefer event-driven handoff between stages; controlled polling is used only for orchestration/status checks (Step Functions Wait, Athena query status — D6+).

01 Idempotency

Replay-safe handlers. Same event arriving twice produces the same end-state, never doubled work.

02 Decoupling

Producers and consumers don't know each other. They know the bus. Replacing one doesn't break the other.

03 Backpressure & retries

Failures get retried with limits. Persistent failures land in DLQ-style destinations for human review.

Your Day 1 prompt becomes today's AWS architecture

The architecture you scored and chose yesterday isn't hypothetical. Every box maps to a service we just covered.

YESTERDAY'S DECISION · /reviews/m1-scorecard.csv → Decision MR

Option A: S3 (raw → validated → curated zones) · Lambda · Glue Catalog · Athena · CloudWatch

How that maps to today's services

S3 raw / validated / curated / quarantine	**S3**	Four prefixes in one bucket. Lab 2 sets up the layout.
Validator on PUT	**Lambda + S3 events**	Lab 3 deploys the starter handler that fires on raw/ PUTs.
IAM execution role	**IAM**	Read raw/, write validated/ + quarantine/. No PII access.
Logs + metrics	**CloudWatch**	Structured JSON, correlation_id, airline-aware fields. Lab 3 reads them.
Catalog + Athena query	**Glue + Athena**	Out of scope today; covered in Module 6.

Today's job: build the first three rows. Module 5 onwards picks up the rest.

PART 1 WRAP-UP

Five things to take into the lab

1. **Five services do the work — IAM, S3, Lambda, CloudWatch, EventBridge.**

   Know the shape of each one. Composition is what changes module to module.

2. **IAM is four primitives + one rule.**

   Identity, policy, role, boundary. Least privilege on top — always.

3. **S3 zones map to data quality stages.**

   raw → validated → curated, with quarantine for failures. Today you set this up.

4. **A Lambda is code + runtime + IAM role.**

   Five settings (runtime, memory, timeout, role, cold start) decide everything else.

5. **Push events; don't poll.**

   Idempotent handlers, decoupled producers/consumers, retries with destinations.

Next: guided design — we walk through the S3 zone model in detail and look at the Lambda handler shape you'll deploy.

02

PART 2 · ~35–40 MINUTES

Guided design

Now we go a level deeper. The S3 zone model in detail, IAM boundaries that prevent privilege escalation, the exact Lambda handler shape you'll deploy, and the logging template every function will use from today onwards.

Four zones, four contracts

Each zone has a clear contract: what it stores, who writes, who reads, how long it lives. The contract is the design.

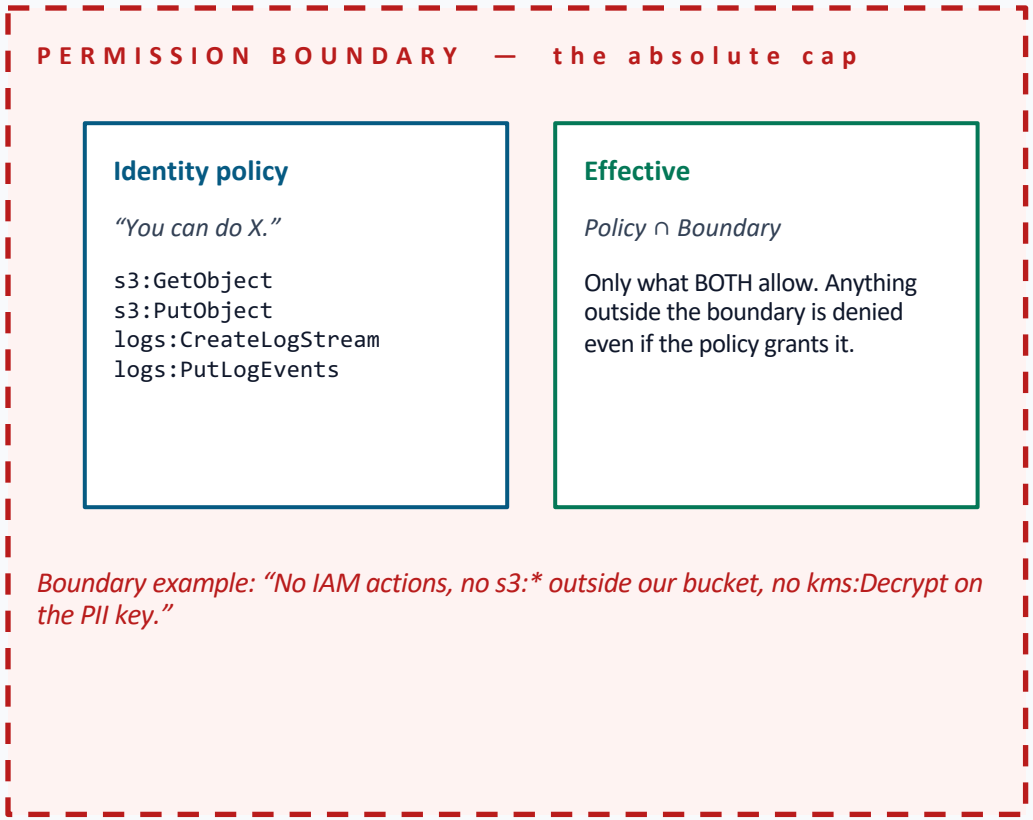
	WRITER	READER	RETENTION	LIFECYCLE
raw/ *Untouched files from the airline. Audit-grade.*	SFTP gateway only	Validator Lambda (read-only)	7 years	Glacier after 90 days
validated/ *Schema-conformant, required fields present.*	Validator Lambda only	Curator Lambda (read-only)	2 years	IA after 30 days
curated/ *Partitioned Parquet. Athena-ready.*	Curator Lambda only	Athena workgroup, dashboards	5 years	IA after 90 days
quarantine/ *Bad records + reason logs. Ops review.*	Validator Lambda only	Ops console role	30 days	Expire after 30 days

Sample workshop contract — final retention depends on client policy. All four prefixes live in one bucket: accelya-airline-data-ap-south-1. IAM is what enforces the contracts above.

Why boundaries matter — the ceiling on what any policy can grant

An attached policy says “you can do X.” A boundary says “no matter what policies attach to you, you can never do beyond Y.”

VISUAL MODEL



01

Policies grant. Boundaries cap.

An identity policy says what you can do. A boundary says how far you can ever go.

02

Both must allow.

Effective permissions are the intersection. If the boundary doesn’t list it, it’s denied.

03

Boundaries protect against escalation.

Even if someone attaches AdministratorAccess to a bounded role, they only get what the boundary allows.

04

Where this shows up here.

Lambda execution roles in this program get a project-wide boundary that excludes IAM writes and the PII KMS key.

Lambda handler anatomy — with an airline payload

Same shape every Lambda you write in this program. Annotations on the right map to the HITL dimensions you score against.

src/validator/handler.py # python3.13 · 256 MB · 30s timeout

```python
import json, os, time
import boto3
from urllib.parse import unquote_plus

s3 = boto3.client("s3")
BUCKET = os.environ["BUCKET_NAME"]
REQUIRED = {"pnr", "airline_code", "fare_class", "od_pair", "settlement_date"}  # lowercase snake_case — same schema as D4-D6

def emit(event_name, **fields):     # JSON log line for CloudWatch Insights
    print(json.dumps({"ts": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "level": "INFO", "event": event_name, **fields}))

def lambda_handler(event, context):
    corr_id = context.aws_request_id
    start = time.time()
    # Lab simplification: one S3 record per upload (loop in production)
    rec = event["Records"][0]["s3"]
    key = unquote_plus(rec["object"]["key"])  # S3 keys are URL-encoded
    emit("received", corr_id=corr_id, function="validator", key=key, zone="raw")

    obj = s3.get_object(Bucket=BUCKET, Key=key)
    rows = json.loads(obj["Body"].read())
    bad = [r for r in rows if not REQUIRED.issubset(r)]
    good = [r for r in rows if REQUIRED.issubset(r)]
    airline_code = rows[0].get("airline_code", "UNKNOWN") if rows else "UNKNOWN"
    # Idempotent + replace prefix only once (count=1)
    s3.put_object(Bucket=BUCKET, Key=key.replace("raw/", "validated/", 1), Body=json.dumps(good).encode("utf-8"),
ContentType="application/json")
    if bad: s3.put_object(Bucket=BUCKET, Key=key.replace("raw/", "quarantine/", 1), Body=json.dumps(bad).encode("utf-8"),
ContentType="application/json")

    emit("validated", corr_id=corr_id, function="validator", airline_code=airline_code, key=key, zone="raw",
         records_total=len(rows), records_good=len(good), records_bad=len(bad),
         duration_ms=int((time.time() - start) * 1000), version=os.getenv("APP_VERSION", "m2"))
    return {"statusCode": 200, "good": len(good), "bad": len(bad)}
```

Correctness

Real APIs only

boto3.s3.put_object — not pseudocode. REQUIRED set lives in code, not a comment.

Security

Bucket from env

BUCKET_NAME via os.environ. No hardcoded ARNs. IAM role provides credentials.

Observability

Correlation ID

context.aws_request_id propagated through every log line. Findable in CloudWatch.

Scalability

Idempotent

Same key replays produce the same end-state. PUT to validated/ is overwrite-safe.

Cost

Bounded

256 MB · 30s timeout · reads one object · writes at most two. No loops.

One JSON shape, every log line, every Lambda

Same fields every function emits. Same parsable shape in CloudWatch. One CloudWatch Insights query finds an entire invocation across services.

log line · JSON, one per call

```
{
  "ts":           "2026-05-05T10:14:32Z",
  "level":        "INFO",
  "corr_id":      "a1b2c3d4-...",
  "function":     "validator",
  "event":        "validated",
  "airline_code": "AI",
  "zone":         "raw",
  "key":          "raw/AI/2026/05/05/file.json",
  "records_total": 482,
  "records_good": 476,
  "records_bad":  6,
  "duration_ms":  284,
  "version":      "v1.2.0"
}
```

Why each field matters

`corr_id`	Trace one file across multiple Lambdas in CloudWatch.
`function + event`	What ran and what stage was reached. Drives metrics.
`airline_code + zone`	Airline-aware queries: “which airline drove yesterday’s quarantine spike?”
`records_*`	Volume + quality counters. Today these are searchable JSON log fields; Module 10 converts selected fields to CloudWatch metrics via EMF.
`duration_ms`	Performance. Outliers reveal cold-start, payload-size, or IO issues.
`version`	Code version that produced the line. Critical when rolling back.

CLOUDWATCH INSIGHTS · ONE INVOCATION END-TO-END

```
fields @timestamp, function, event, records_good, records_bad
| filter corr_id = "a1b2c3d4-..."
| sort @timestamp asc
```

03

PART 3 · ~2–2¼ HOURS

Hands-on labs

Three labs, in order. Lab 1 verifies your IAM identity. Lab 2 builds the S3 zone layout in the sandbox. Lab 3 deploys a real validator Lambda and traces a single invocation through CloudWatch.

LAB 1 · ~30 MINUTES

AWS sandbox sign-in + IAM verification

Confirm who you are in AWS and what you're allowed to do — before we touch any storage or compute.

TIME	DIFFICULTY	MODE	DELIVERABLE
30 min	Foundation	Solo + pair check	m2-iam-verification.md

OBJECTIVE

By the end of this lab, you will have:

- Confirmed your IAM identity in ap-south-1 via the CLI
- Read the policy + permission boundary attached to your role
- Probed an allowed action (S3 read on raw/ — should succeed)
- Probed a denied IAM action (iam:CreateUser — should deny via boundary)
- Documented findings in your branch

PROVISIONED FOR YOU

- **IAM user** accelya-learner-<your-name> with console + CLI access.
- **Lambda execution role** accelya-lambda-role-<your-name> (you assume this in Lab 3).
- **Permission boundary** accelya-program-boundary attached to both.
- **Shared bucket** accelya-airline-data-ap-south-1 with raw/, validated/, curated/, quarantine/.
- **Synthetic test files** in raw/<airline>/2026/05/ for Labs 2 and 3.

Four steps. Twenty minutes if everything works.

1 Confirm identity

```
aws sts get-caller-identity --region ap-south-1
```

EXPECTED

JSON returns Account, Arn, UserId. Arn ends in your accelya-learner-<name>.

2 Read identity + boundary

```
aws iam get-user --user-name accelya-learner-<your-name>
```

EXPECTED

JSON shows PermissionsBoundary attribute pointing at accelya-program-boundary. (If your sandbox issues an assumed role / federated identity instead of an IAM user, get-user will fail with NoSuchEntity — fall back to: aws iam get-role --role-name <role-from-sts-arn> to inspect PermissionsBoundary, then aws iam list-

3 Probe an allowed action

```
aws s3 ls s3://accelya-airline-data-ap-south-1/raw/ --region ap-south-1
```

EXPECTED

Subfolders for sample airlines: AI/, BA/, EK/, LH/, SQ/. No AccessDenied.

4 Probe a denied action

```
aws iam create-user --user-name should-fail   # IAM is global — no --region
```

EXPECTED

AccessDeniedException — boundary blocks IAM writes. Capture the error message.

Success criteria & common gotchas

✓ YOU ARE READY FOR LAB 2 IF…

- **get-caller-identity returns your accelya-learner ARN** in ap-south-1.
- **PermissionsBoundary visible in the get-user JSON output, pointing at accelya-program-boundary. (Assumed-role sandbox: get-role --role-name <role-from-sts-arn> confirms PermissionsBoundary instead.)**
- **Allowed S3 list succeeds** against raw/ in the shared bucket.
- **Denied IAM action fails cleanly** — you saw the AccessDeniedException, not a network error.
- **Findings committed** to /reviews/m2-iam-verification.md on your branch.

⚠ TOP FIVE GOTCHAS — CHECK FIRST

Wrong region

AWS_DEFAULT_REGION not set → commands hit us-east-1, sandbox blocks them. Set ap-south-1 or pass --region.

Stale credentials

aws configure was run with old keys. sts call returns InvalidClientTokenId. Reissue from sandbox console.

Wrong user name

iam get-user expects exact case. Use the name from sts get-caller-identity Arn.

Misreading deny

If create-user succeeds, the boundary is wrong — stop and tell trainer. It must deny.

Pasting the IAM ARN into prompts

Treat ARNs as sensitive. Paste excerpts, not full identifiers, into shared docs.

LAB 2 · ~45 MINUTES

S3 zones — inspect and populate

Walk the four zones in the shared bucket. Drop a synthetic ticket-issue file into your learner prefix. Confirm the layout matches the contract from slide 15.

TIME	DIFFICULTY	MODE	DELIVERABLE
45 min	All bands	Pairs	m2-s3-layout.md

YOUR SYNTHETIC TEST FILE — ALREADY IN /data/synth/

ticket-issues-AI-2026-05-05.json — 500 records, 6 deliberately malformed (missing fare_class), 494 clean.

What you'll do

Inspect — List all four zones. Check existing data in raw/, count objects per airline.

Document — Sketch the bucket layout in m2-s3-layout.md — prefixes, retention, who writes.

Place — Stage the synthetic file under learner-sandbox/<your-name>/raw-staging/2026/05/05/. Lab 3 is the first upload to raw/ (which fires the trigger).

Verify — List your prefix back. Confirm size and timestamp.

Five steps. Thirty minutes if everything works.

1 List the bucket layout

```
aws s3 ls s3://accelya-airline-data-ap-south-1/ --region ap-south-1
```

EXPECTED

Four prefixes: raw/, validated/, curated/, quarantine/. No surprises.

2 Count raw objects per airline

```
aws s3 ls s3://accelya-airline-data-ap-south-1/raw/AI/ --recursive \
  --summarize --region ap-south-1 | tail -2
```

EXPECTED

Total Objects + Total Size print. Sample airlines: AI ~5 files, BA ~3, EK ~7.

3 Upload synthetic file to your prefix

```
aws s3 cp /data/synth/ticket-issues-AI-2026-05-05.json \
  s3://accelya-airline-data-ap-south-1/learner-sandbox/<your-name>/raw-staging/2026/05/05/ \
  --region ap-south-1
```

EXPECTED

upload: ./ticket-issues-AI-2026-05-05.json to s3://... — size matches local.

4 Verify your prefix

```
aws s3 ls s3://accelya-airline-data-ap-south-1/learner-sandbox/<your-name>/raw-staging/ \
  --recursive --human-readable --region ap-south-1
```

EXPECTED

One file listed with correct size and recent timestamp.

5 Commit your layout doc

```
git add docs/m2-s3-layout.md && git commit -m "m2: s3 layout documented" \
  && git push origin module-2-<your-name>
```

EXPECTED

Branch pushed. Open MR in GitLab (or use git push -o merge_request.create). HITL_REVIEW.md pending.

Success criteria & common gotchas

✓ YOU ARE READY FOR LAB 3 IF…

- **Four zones visible** under the shared bucket — raw, validated, curated, quarantine.
- **Synthetic file in learner-sandbox/<your-name>/raw-staging/ (NOT raw/ — that's Lab 3's trigger zone) with the right size, partition path, and timestamp.**
- **raw/<your-name>/, validated/<your-name>/, and quarantine/<your-name>/ ALL still empty — Lab 3 is the first one to write into raw/ (and the trigger fires).**
- **m2-s3-layout.md committed** with the contract for each zone (writer/reader/retention/lifecycle).
- **Branch pushed** to GitLab; CI lint job is green.

⚠ FIVE GOTCHAS — CHECK FIRST

Wrong prefix path

Forgetting <your-name>/ writes into a peer's area. Lab 2 stages under learner-sandbox/<your-name>/; do NOT upload to raw/ in Lab 2 — that fires Lab 3's trigger early. Use --dryrun to verify.

Trailing slash

Without a trailing slash, S3 may treat the destination as an object name, not a prefix.

AccessDenied on validated/

Expected — your user can't write directly there. Lab 3 deploys the Lambda that does.

Local file path

/data/synth/ is the trainer-mounted dir on the lab box; on your laptop it lives elsewhere.

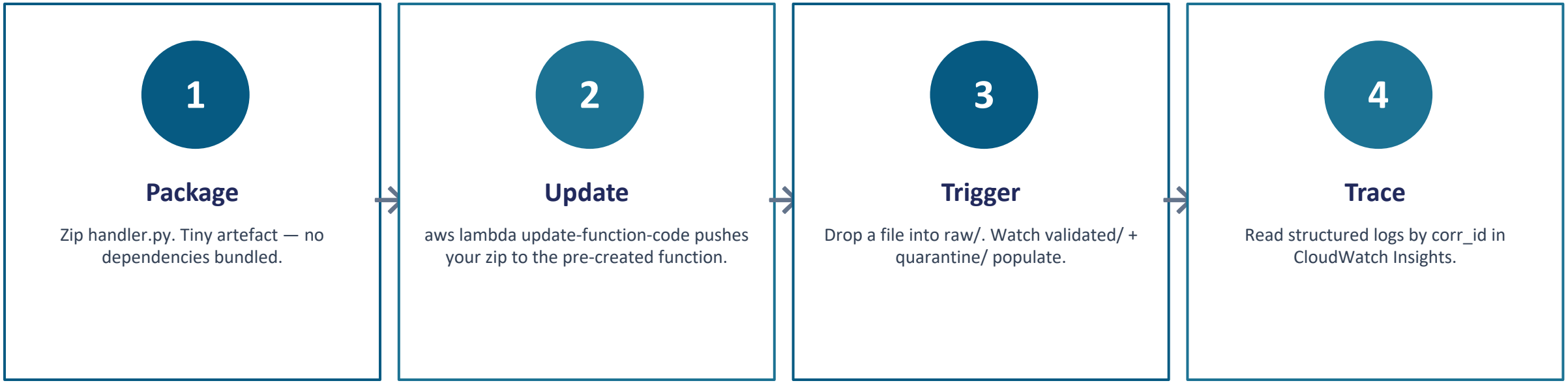
Versioning surprise

PUTting twice creates two versions. ls --recursive shows the newest; check VersionId if you need older.

Deploy the validator Lambda — trace one invocation end-to-end

Take the handler from slide 17. Package it. Push it to the pre-created function. Trigger it. Read the logs. Score it against the HITL checklist.

TIME	DIFFICULTY	MODE	DELIVERABLE
60 min	Working+	Pairs	running Lambda + HITL_REVIEW.md



⚠ **PRE-PROVISIONED BY THE TRAINER**

The Lambda skeleton, IAM execution role, S3 bucket notification, and `lambda:add-permission` are all pre-provisioned. You package and push code via `update-function-code`. Trigger is scoped to `raw/<learner>/` only — writes to validated/ and quarantine/ never re-fire the Lambda.

The actual commands you'll run

1 Package the function

```
cd src/validator && zip -j /tmp/validator.zip handler.py && cd -
```

EXPECTED

/tmp/validator.zip created. < 5 KB. Contains handler.py at root.

2 Update code on the pre-created function

```
aws lambda update-function-code \
  --function-name accelya-m2-validator-<your-name> \
  --zip-file fileb:///tmp/validator.zip --region ap-south-1
```

EXPECTED

LastUpdateStatus=Successful. Run aws lambda wait function-updated-v2 --function-name <fn> to confirm the code update is complete before triggering.

3 Trigger by dropping a file into raw/

```
aws s3 cp /data/synth/ticket-issues-AI-2026-05-05.json \
  s3://accelya-airline-data-ap-south-1/raw/<your-name>/2026/05/05/ \
  --region ap-south-1
```

EXPECTED

This is the FIRST upload into the trigger zone raw/<your-name>/ (Lab 2 staged into learner-sandbox/). Within seconds: validated/<your-name>/... appears with 494 records; quarantine/<your-name>/... with 6.

4 Tail the structured logs

```
aws logs tail /aws/lambda/accelya-m2-validator-<your-name> \
  --follow --format short --region ap-south-1
```

EXPECTED

JSON lines: "event":"received" then "event":"validated". Note the corr_id.

5 Find the full invocation in Insights

```
# Cross-platform UTC epoch seconds (works on macOS + Linux):
START=$(python3 -c 'import time; print(int(time.time()-600))')
END=$(python3 -c 'import time; print(int(time.time()))')
aws logs start-query --log-group-name /aws/lambda/accelya-m2-validator-<your-name> \
  --start-time $START --end-time $END \
  --query-string 'fields @timestamp, event, records_good, records_bad | filter corr_id = "<id>"'
```

EXPECTED

QueryId returned. Poll until status=Complete:
aws logs get-query-results --query-id <QueryId> --region ap-south-1

S3 → Lambda trigger is pre-provisioned by the trainer with the notification filter scoped to raw/<learner>/ ONLY (writes to validated/ and quarantine/ do NOT re-fire — same-bucket recursion guarded by the prefix filter). If step 3 doesn't fire, ask in chat — don't change the bucket policy.

HITL checklist — against your running Lambda

Same five dimensions as Day 1 (programme rule: 0-3 per dim · PASS ≥12/15 + every dim ≥2). Now you're scoring real code on a real cloud, not a Bedrock-generated proposal.

/reviews/m2-validator-scorecard.csv

DIM	SCORE	EVIDENCE
Correctness	**3**	494 good / 6 bad as expected. Schema check matches REQUIRED set.
Security	**2**	IAM role tight; bucket from env. Boundary verified in Lab 1.
Observability	**3**	corr_id traces every step. Insights query works.
Scalability	**2**	Idempotent on replay; tested with same key twice.
Cost	**2**	256 MB / 30s feasible. Could right-size to 192 MB after profiling.

✓ YOUR LAB 3 IS COMPLETE WHEN

- **Lambda function updated** in ap-south-1; State=Active, LastUpdateStatus=Successful.
- **S3 PUT triggers it** — validated/ and quarantine/ populate within seconds.
- **Logs show structured JSON** with corr_id, records_good, records_bad, duration_ms.
- **Insights query returns the full invocation** for one file.
- **HITL_REVIEW.md filled honestly** — unmet items documented, not hidden.
- **MR opened** on GitLab. Trainer reviews live.

What you just built — in one frame

LAB 1

IAM verified

Identity, policies, and boundary inspected and probed in ap-south-1

ARTEFACT

`/reviews/m2-iam-verification.md`

LAB 2

S3 layout placed

Synthetic file staged in learner-sandbox/<your-name>/raw-staging/; raw/ trigger zone is used in Lab 3

ARTEFACT

`/docs/m2-s3-layout.md`

LAB 3

Lambda live

Deployed validator processing files end-to-end; HITL-scored

ARTEFACT

`Running Lambda + scorecard MR`

Next: 20–30 minute wrap. Common IAM mistakes, end-of-module cleanup, Day 3 preview — the prompt-engineering deep-dive on top of what you just built.

04

PART 4 · ~20–30 MINUTES

Review, Q&A, wrap

Five IAM and Lambda mistakes worth naming. End-of-module cleanup (you actually have a Lambda to tear down today). Day 3 preview — the prompt-engineering deep-dive on top of what you just shipped.

Five HITL mistakes from today — and what to do instead

These are the patterns AI-generated AWS code falls into. Reviewers (you, the rest of the cohort, future you) need to spot them on sight.

01 Wildcard IAM policies

THE MISTAKE

s3:* on the bucket; iam:* on "Resource": "*"

DO INSTEAD

Name the actions. Scope the ARNs. raw/* and validated/* are different policies.

02 Hardcoded bucket name

THE MISTAKE

BUCKET = "accelya-airline-data-ap-south-1" baked into handler.py

DO INSTEAD

BUCKET = os.environ["BUCKET_NAME"]. Set per environment in deploy config.

03 No correlation ID in logs

THE MISTAKE

log.info("validating") with nothing to tie it to a specific invocation

DO INSTEAD

Always emit JSON logs with corr_id, function, event, and key. Insights queries depend on it.

04 Synchronous loops in handler

THE MISTAKE

for record in 1000_records: boto3.client(...).invoke(...) — serial waits

DO INSTEAD

Batch where possible; use SQS or Step Functions for fan-out. Lambda timeout is 15 min, not 15 hours.

05 Skipping HITL_REVIEW.md

THE MISTAKE

Code merges with the scorecard blank or dimensions left as “TBD”

DO INSTEAD

Fill it before pushing. “Cost: 3 — 256 MB feasible, will profile after Module 5.” Honest beats absent.

Module 3 onwards: every code review you do (peer or AI-assisted) starts with these five.

End-of-module cleanup

Reset your code on the trainer-managed Lambda. Empty your S3 prefixes. Leave the function and log group in place — Day 3 builds on them.

scripts/cleanup_module2.sh # Linux/WSL example — macOS/Windows: use verify_cleanup.sh

```
# 1. Reset learner code on the pre-created Lambda
#    (do NOT delete the function — trainer keeps it for Day 3)
./scripts/reset_module2_lambda.sh $LEARNER

# 2. Keep CloudWatch log group — logs are reviewed in Day 3
#    (no delete-log-group; trainer manages retention)

# 3. Empty Lab 2 staging prefix (learner-sandbox/<learner>/raw-staging/)
aws s3 rm s3://accelya-airline-data-ap-south-1/learner-sandbox/$LEARNER/ \
  --recursive --region ap-south-1

# 4. Empty your learner prefixes — raw/, validated/, quarantine/
for ZONE in raw validated quarantine; do
  aws s3 rm s3://accelya-airline-data-ap-south-1/$ZONE/$LEARNER/ \
    --recursive --region ap-south-1
done

# 5. Push branch & open MR on GitLab
git push origin module-2-$LEARNER

# 6. Verify cleanup
./verify_cleanup.sh    # expects: ALL CLEAR
```

Cleanup checklist

- [ ] Pre-created Lambda remains present (do NOT delete)
- [ ] Function code reset to baseline; ready for Day 3
- [ ] CloudWatch log group preserved for review
- [ ] Your raw/, validated/, quarantine/, AND learner-sandbox/<your-name>/raw-staging/ prefixes all empty
- [ ] All artefacts committed to module-2-<your-name>
- [ ] HITL_REVIEW.md filled and committed
- [ ] GitLab MR opened
- [ ] verify_cleanup.sh prints ALL CLEAR

⚠ **COST GUARDRAIL**

Pluralsight sandbox caps are per learner — leaving files in raw/<your-name>/ can re-trigger the Lambda and burn Lambda invocations, S3 storage, and CloudWatch ingest. (D2 Lambda doesn't call Bedrock, so the Bedrock invocation budget is unaffected here.) Empty your prefixes.

Day 2 in numbers

Snapshot of what every learner walks out of Module 2 with. This is the running baseline for everything Module 3 onwards builds on.

1

Lambda

deployed and traced end-to-end

4

S3 zones

inspected, populated, contracted

5

HITL dims

scored against running code

5

AWS services

covered in concept + lab

12

terms

of airline vocabulary reused in payloads + logs

ARTEFACTS COMMITTED TO YOUR GITLAB BRANCH TODAY

- `/reviews/m2-iam-verification.md`
  policy + boundary findings (Lab 1)
- `/docs/m2-s3-layout.md`
  bucket layout + zone contracts (Lab 2)
- `src/validator/handler.py`
  the function actually deployed (Lab 3)
- `/reviews/m2-validator-scorecard.csv`
  5-dimension HITL score with evidence
- `HITL_REVIEW.md`
  module 2 row filled
- `Decision MR on GitLab`
  trainer review pending

Day 3 preview — prompt engineering, on top of working AWS

Module 1 introduced the HITL flow. Module 2 built the AWS layer it lands on. Module 3 goes deeper into the agentic craft itself.

DAY 3 STRUCTURE · 4 HOURS

30–35 min

Concepts

Context engineering · Repo-aware prompting · Failure modes (hallucination, confident wrong)

35–40 min

Guided design

Writing .windsurfrules that work · Prompt injection deep-dive · When to load files vs explain

2–2¼ hrs

Labs

Prompt your validator to write its own tests · Detect a planted prompt-injection · AI-assisted code review

20–30 min

Review & wrap

Score AI-generated tests against HITL · Cleanup · Day 4 preview (batch ingestion deep-dive)

HOW DAY 3 BUILDS ON DAY 2

Day 2 → Day 3

Your running validator Lambda

→ *becomes the input to a test-generation prompt.*

Your structured logs

→ *become reference material for context-aware prompting.*

Your IAM boundary

→ *becomes a constraint AI must respect when proposing changes.*

The HITL checklist

→ *gets applied to AI-generated tests — not just AI-generated architectures.*

The .windsurfrules from Day 1

→ *gets rewritten with Day 2 lessons baked in ("never hardcode bucket", "always corr_id").*

MODULE 2 COMPLETE

You shipped a real Lambda.

On a real cloud.
Observable. Bounded. Scored.

IAM identity verified

S3 zones populated

Lambda deployed + HITL-scored

Tomorrow we make the agent itself the subject — prompts, tests, failure modes. Same repo. New module branch. Now with a deployed Lambda to point at.